

React Interview Answers

Technical Questions

1. Explain React's component lifecycle methods (class components) vs. hooks (functional components).

Class Components Lifecycle:

- **Mounting Phase:** `constructor()`, `static getDerivedStateFromProps()`, `render()`, and `componentDidMount()`
- **Updating Phase:** `static getDerivedStateFromProps()`, `shouldComponentUpdate()`, `render()`, `getSnapshotBeforeUpdate()`, and `componentDidUpdate()`
- **Unmounting Phase:** `componentWillUnmount()`
- **Error Handling:** `static getDerivedStateFromError()` and `componentDidCatch()`

Functional Components with Hooks:

- `useState`: Replaces state management in class components
- `useEffect`: Combines functionality of `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`
- `useContext`: Accesses context without consumer components
- `useReducer`: Alternative to `useState` for complex state logic
- `useRef`: Persists values between renders without causing re-renders
- `useMemo` and `useCallback`: Optimize performance by memoizing values and functions
- `useLayoutEffect`: Similar to `useEffect` but fires synchronously after DOM mutations

Key Differences:

- Hooks provide a more concise API and help avoid the confusion of lifecycle methods
- Hooks enable better code organization by grouping related logic together
- Functional components with hooks simplify testing and improve type checking
- Hooks enable code reuse through custom hooks, eliminating the need for HOCs or render props

2. How do you optimize performance in React applications?

Component-Level Optimizations:

- Use `React.memo` for functional components to prevent unnecessary re-renders
- Implement `shouldComponentUpdate` or `PureComponent` for class components

- Use `useCallback` to memoize functions passed as props
- Implement `useMemo` to cache expensive calculations
- Leverage `useRef` for values that don't trigger re-renders

Code-Splitting and Lazy Loading:

- Use dynamic imports and `React.lazy` to split code into smaller chunks
- Implement `Suspense` for loading states during lazy loading
- Prioritize above-the-fold content loading

Efficient Rendering:

- Use virtualization libraries like `react-window` or `react-virtualized` for long lists
- Implement windowing techniques for large data sets
- Keep component state as local as possible
- Avoid anonymous functions in render methods

State Management:

- Normalize complex state structures
- Use immutable data patterns
- Consider using more efficient state management libraries for large applications

Build and Bundle Optimizations:

- Implement tree shaking to eliminate dead code
- Configure proper code splitting in Webpack
- Use production builds with minification
- Optimize asset loading with proper caching strategies

3. Explain the virtual DOM and reconciliation algorithm.

Virtual DOM:

The Virtual DOM is a lightweight JavaScript representation of the actual DOM. React creates and maintains this virtual representation, which allows it to compute differences before updating the real DOM.

Key Aspects:

- It's a programming concept, not a specific technology
- React elements are plain JavaScript objects describing UI components
- Changes are first applied to the virtual DOM, not the real DOM

- React can batch multiple updates together for efficiency

Reconciliation Algorithm:

Reconciliation is the process React uses to determine what parts of the actual DOM need to be updated.

How It Works:

1. When state or props change, React creates a new virtual DOM tree
2. React compares the new virtual tree with the previous one using a diffing algorithm
3. It identifies the minimal set of operations needed to update the real DOM
4. React applies only these necessary changes to the actual DOM

Key Optimizations:

- React uses heuristics that assume:
 - Elements of different types produce different trees
 - Developer-supplied keys help identify which child elements may be stable across renders
- The algorithm has $O(n)$ complexity rather than $O(n^3)$ of naive diffing
- React only updates DOM nodes when necessary, reducing expensive DOM manipulations

Fiber:

React 16 introduced Fiber, a complete rewrite of the reconciliation algorithm that allows:

- Work to be split into chunks and prioritized
- Work to be paused, reused, or aborted
- Different types of updates to be assigned different priorities
- Better support for error boundaries and concurrent mode

4. How do you manage state in React? Compare useState, useReducer, Context API, and external state management.

useState:

- Best for simple, independent state values
- Provides a current state value and a function to update it
- Updates are merged rather than replaced (unlike class component setState)
- Example: `const [count, setCount] = useState(0);`

useReducer:

- Preferable for complex state logic with multiple sub-values
- Uses a reducer function (state, action) => newState

- Enables more predictable state transitions
- Helps centralize related state transitions
- Example: `const [state, dispatch] = useReducer(reducer, initialState);`

Context API:

- Provides a way to share state across components without prop drilling
- Best for global state that doesn't change frequently
- Creates a provider/consumer relationship
- Can be combined with useState or useReducer for more complex state
- Performance concerns with frequent updates affecting all consumers
- Example: `const value = useContext(MyContext);`

External State Management (Redux, MobX, Zustand, Recoil, Jotai):

- Redux: Predictable state container with a single store, reducers, and middleware
- MobX: Observable-based state management with less boilerplate than Redux
- Zustand: Minimalist approach with a simple, hook-based API
- Recoil: Atomic state management designed specifically for React
- Jotai: Primitive and flexible state management inspired by Recoil

Selection Criteria:

- Application size and complexity
- Team familiarity with the solution
- Need for time-travel debugging
- Performance requirements
- Development experience preferences
- Need for middleware (side effects, persistence, etc.)

Best Practices:

- Start with local state (useState/useReducer)
- Use Context for moderate-sized applications
- Consider external libraries for large, complex applications
- Combine approaches as needed (e.g., Context + useReducer)

5. Describe your experience with Redux, Redux Toolkit, MobX, Zustand, or other state management libraries.

I have extensive experience with multiple state management solutions, each with their distinct advantages:

Redux:

- Implemented in enterprise applications with complex workflows
- Leveraged middleware for async operations (redux-thunk, redux-saga)
- Integrated with React Router for route-based state management
- Used selectors for efficient state derivation
- Implemented normalization patterns for relational data

Redux Toolkit:

- Adopted to reduce boilerplate code in Redux applications
- Used createSlice for defining reducers and actions
- Implemented createAsyncThunk for standardized async operations
- Leveraged RTK Query for data fetching and caching
- Configured optimistic updates for improved user experience

MobX:

- Implemented in applications requiring real-time updates and complex object graphs
- Used decorators and makeObservable for declaring observable state
- Leveraged computed values for derived state
- Implemented reactions for side effects
- Used strict mode to enforce action usage for state mutations

Zustand:

- Adopted for smaller applications requiring simpler state management
- Utilized middleware for persistence and logging
- Implemented custom selectors for performance optimization
- Integrated with React Query for server state management

Context + useReducer:

- Implemented for medium-sized applications
- Created custom provider components for modular state
- Used context selectors to prevent unnecessary re-renders
- Integrated with local storage for persistence

6. How do you handle side effects in React?

Using useEffect Hook:

- For lifecycle events like component mount and unmount
- For subscriptions to external data sources
- For data fetching operations
- For DOM manipulations
- Example:

jsx

 Copy

```
useEffect(() => {  
  const subscription = dataSource.subscribe();  
  return () => subscription.unsubscribe(); // Cleanup  
}, [dataSource]); // Dependency array
```

Dependency Management:

- Properly managing the dependency array is crucial
- Empty array (`[]`) means "run once on mount"
- No dependency array means "run on every render"
- Including dependencies means "run when these values change"

Cleanup Functions:

- Return a function from `useEffect` for cleanup operations
- Used for unsubscribing, clearing timers, or removing event listeners
- Runs before the component unmounts or before the effect runs again

Effect Timing:

- `useEffect` runs after render is committed to the screen
- `useLayoutEffect` runs synchronously after DOM mutations but before the browser paints

Custom Hooks:

- Extract common effect patterns into reusable custom hooks
- Promotes code reuse and maintainability
- Example: `useDataFetching`, `useLocalStorage`, `useWindowSize`

Alternative Approaches:

- Redux middleware (thunks, sagas) for state-related side effects

- React Query, SWR for data fetching with caching
- Event-driven architectures using libraries like RxJS
- Custom event systems for complex workflows

Best Practices:

- Keep effects focused on a single responsibility
- Use cleanup functions to prevent memory leaks
- Split complex effects into multiple hooks
- Consider using a library for complex async flows

7. What are React Error Boundaries and how do you implement them?

Error Boundaries:

Error boundaries are special React components that catch JavaScript errors in their child component tree, log these errors, and display a fallback UI instead of crashing the entire app.

Key Characteristics:

- Only catch errors in their children, not in the error boundary itself
- Don't catch errors in event handlers, asynchronous code, or server-side rendering
- Only work in class components, not functional components

Implementation:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    // Update state to show fallback UI
    return { hasError: true };
  }

  componentDidCatch(error, errorInfo) {
    // Log the error to an error reporting service
    logErrorToService(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      // Render fallback UI
      return <h1>Something went wrong.</h1>;
    }

    return this.props.children;
  }
}

// Usage
<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>
```

Best Practices:

- Use multiple error boundaries to isolate different parts of the application
- Implement more granular error boundaries for critical components
- Create reusable error boundary components with customizable fallback UIs
- Integrate with error monitoring services
- Consider state recovery options after an error occurs

With Hooks:

- While error boundaries must be class components, you can create a custom hook to simplify usage:


```
// Use with a class component error boundary
function useErrorBoundary() {
  const [error, setError] = useState(null);

  if (error) {
    throw error; // This will be caught by the nearest error boundary
  }

  return setError;
}
```

8. Explain React Server Components and their benefits.

React Server Components:

Introduced in React 18, Server Components allow developers to build applications that span the server and client, combining the rich interactivity of client-side apps with the improved performance of traditional server rendering.

Key Characteristics:

- Server Components run only on the server and never on the client
- They can access server-side resources directly (databases, file systems)
- Their code isn't included in the JavaScript bundle sent to clients
- They can be used alongside traditional Client Components

Component Types:

- **Server Components:** Run only on the server, can't use hooks or browser APIs
- **Client Components:** Traditional React components that run in the browser
- **Shared Components:** Components that can run in both environments

Benefits:

- **Reduced Bundle Size:** Server component code stays on the server
- **Direct Access to Data Sources:** No need for client-side data fetching
- **Improved Performance:** Faster initial load and reduced client-side processing
- **Simplified Data Fetching:** No need for `useEffect` for initial data
- **Progressive Enhancement:** Servers can render complex UI while clients handle interactivity
- **Improved Security:** Sensitive code and data access remain on the server

Implementation with Next.js:

```
// Server Component (app/page.js)
async function BlogPosts() {
  // Direct database access
  const posts = await db.posts.findMany();

  return (
    <div>
      {posts.map(post => <PostPreview key={post.id} post={post} />)}
    </div>
  );
}

// Client Component (with "use client" directive)
'use client'
function LikeButton({ postId }) {
  const [liked, setLiked] = useState(false);

  return (
    <button onClick={() => setLiked(!liked)}>
      {liked ? 'Liked' : 'Like'}
    </button>
  );
}
```

Limitations:

- Server Components can't use React hooks
- They can't access browser-specific APIs
- They require a compatible framework like Next.js to implement
- They introduce a new mental model for component boundaries

9. How do you implement code splitting in React?

Code Splitting:

Code splitting is a technique that breaks down your application bundle into smaller chunks that can be loaded on demand, improving initial load performance.

Using Dynamic Import:

```
import { useState } from 'react';

function MyComponent() {
  const [module, setModule] = useState(null);

  const loadModule = async () => {
    const mod = await import('./heavy-module');
    setModule(mod);
  };

  return (
    <div>
      <button onClick={loadModule}>Load Module</button>
      {module && <module.default />}
    </div>
  );
}
```

Using React.lazy and Suspense:

```
import React, { Suspense, lazy } from 'react';

const HeavyComponent = lazy(() => import('./HeavyComponent'));

function MyApp() {
  return (
    <div>
      <Suspense fallback={<div>Loading...</div>}>
        <HeavyComponent />
      </Suspense>
    </div>
  );
}
```

Route-Based Code Splitting:

```
import React, { Suspense, lazy } from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';

const Home = lazy(() => import('./routes/Home'));
const About = lazy(() => import('./routes/About'));
const Contact = lazy(() => import('./routes/Contact'));

function App() {
  return (
    <Router>
      <Suspense fallback={<div>Loading...</div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/contact" element={<Contact />} />
        </Routes>
      </Suspense>
    </Router>
  );
}
```

Component-Level Code Splitting:

```
const ModalComponent = lazy(() => import('./ModalComponent'));

function App() {
  const [showModal, setShowModal] = useState(false);

  return (
    <div>
      <button onClick={() => setShowModal(true)}>Open Modal</button>
      {showModal && (
        <Suspense fallback={<div>Loading modal...</div>}>
          <ModalComponent onClose={() => setShowModal(false)} />
        </Suspense>
      )}
    </div>
  );
}
```

Best Practices:

- Focus on splitting at natural boundaries (routes, modals, tabs)
- Use analytics to identify unused code areas for splitting
- Implement prefetching for predictable user journeys
- Balance number of chunks (too many small chunks can degrade performance)
- Test performance on lower-end devices and slower networks
- Use webpack bundle analyzer to identify splitting opportunities

10. Describe your experience with React hooks. Which custom hooks have you created?

I've extensively used React hooks since their introduction in React 16.8, transforming how I approach component development:

Built-in Hooks Usage:

- `useState` for local component state
- `useEffect` for side effects, data fetching, and subscriptions
- `useContext` for consuming context values
- `useReducer` for complex state logic
- `useCallback` and `useMemo` for performance optimizations
- `useRef` for DOM references and persisting values
- `useLayoutEffect` for synchronous DOM measurements
- `useImperativeHandle` for customizing instance values with `forwardRef`

Custom Hooks I've Created:

1. Data Fetching:

```
function useFetch(url, options = {}) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isMounted = true;

    async function fetchData() {
      try {
        setLoading(true);
        const response = await fetch(url, options);

        if (!response.ok) {
          throw new Error(`HTTP error ${response.status}`);
        }

        const result = await response.json();
        if (isMounted) {
          setData(result);
          setError(null);
        }
      } catch (err) {
        if (isMounted) {
          setError(err.message);
          setData(null);
        }
      } finally {
        if (isMounted) {
          setLoading(false);
        }
      }
    }

    fetchData();

    return () => {
      isMounted = false;
    };
  }, [url, JSON.stringify(options)]);

  return { data, loading, error };
}
```

2. Local Storage Integration:

jsx

 Copy

```
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.error(error);
      return initialValue;
    }
  });

  const setValue = (value) => {
    try {
      const valueToStore = value instanceof Function ? value(storedValue) : value;
      setStoredValue(valueToStore);
      window.localStorage.setItem(key, JSON.stringify(valueToStore));
    } catch (error) {
      console.error(error);
    }
  };

  return [storedValue, setValue];
}
```

3. Form Management:

```
function useForm(initialValues = {}) {
  const [values, setValues] = useState(initialValues);
  const [errors, setErrors] = useState({});

  const handleChange = (e) => {
    const { name, value } = e.target;
    setValues({
      ...values,
      [name]: value
    });
  };

  const validate = (validationSchema) => {
    const newErrors = {};
    Object.keys(validationSchema).forEach(field => {
      const value = values[field] || '';
      const fieldErrors = validationSchema[field]
        .filter(validator => !validator.fn(value))
        .map(validator => validator.message);

      if (fieldErrors.length > 0) {
        newErrors[field] = fieldErrors[0];
      }
    });

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  const reset = () => setValues(initialValues);

  return {
    values,
    errors,
    handleChange,
    reset,
    validate
  };
}
```

4. Media Queries:


```
function useMediaQuery(query) {
  const [matches, setMatches] = useState(
    () => window.matchMedia(query).matches
  );

  useEffect(() => {
    const mediaQuery = window.matchMedia(query);
    const handleChange = (e) => setMatches(e.matches);

    mediaQuery.addEventListener('change', handleChange);
    return () => mediaQuery.removeEventListener('change', handleChange);
  }, [query]);

  return matches;
}
```

5. Debounce:

```
function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}
```

11. How do you handle form state in React?

Controlled Components:

```
function ControlledForm() {
  const [formState, setFormState] = useState({
    name: '',
    email: '',
    message: ''
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormState(prev => ({
      ...prev,
      [name]: value
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(formState);
    // Submit logic
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="name"
        value={formState.name}
        onChange={handleChange}
      />
      { /* Other fields */ }
      <button type="submit">Submit</button>
    </form>
  );
}
```

Uncontrolled Components with Refs:

```
function UncontrolledForm() {
  const nameRef = useRef();
  const emailRef = useRef();
  const messageRef = useRef();

  const handleSubmit = (e) => {
    e.preventDefault();
    const formData = {
      name: nameRef.current.value,
      email: emailRef.current.value,
      message: messageRef.current.value
    };
    console.log(formData);
    // Submit logic
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" ref={nameRef} defaultValue="" />
      {/* Other fields */}
      <button type="submit">Submit</button>
    </form>
  );
}
```

Form Libraries:

- **Formik:** Comprehensive form management with validation
- **React Hook Form:** Performance-focused with uncontrolled components
- **Redux Form:** For Redux-integrated applications
- **Final Form:** Subscription-based form state

React Hook Form Example:

```
import { useForm } from 'react-hook-form';

function HookForm() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  const onSubmit = (data) => {
    console.log(data);
    // Submit logic
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input
        {...register('name', { required: 'Name is required' })}
      />
      {errors.name && <p>{errors.name.message}</p>}
      { /* Other fields */ }
      <button type="submit">Submit</button>
    </form>
  );
}
```

Form Validation Approaches:

- **HTML5 Validation:** Using built-in attributes (required, pattern, min, max)
- **JavaScript Validation:** Custom validation logic in event handlers
- **Schema Validation:** Using libraries like Yup, Zod, or Joi
- **Server-side Validation:** Always validate on the server regardless of client-side validation

Best Practices:

- Keep form state as local as possible
- Implement proper error handling and validation
- Use field-level validation for immediate feedback
- Implement form-level validation for cross-field validation
- Provide clear error messages
- Handle disabled states during submission
- Implement proper keyboard navigation and accessibility

12. Explain React's rendering and re-rendering behavior.

Initial Rendering Process:

1. React creates a virtual DOM representation from JSX
2. Renders all components recursively beginning at the root
3. Converts the virtual DOM to actual DOM instructions
4. Updates the browser DOM (mounting)

Re-rendering Triggers:

- State changes via `setState` or state updater functions
- Parent component re-renders (props potentially change)
- Context value changes for consuming components
- Hooks like `useMemo`, `useCallback` with changed dependencies
- Force updates (rarely used)

Render Phase:

- React calls component functions to get their JSX representation
- Creates a new virtual DOM tree
- Runs the reconciliation algorithm to determine what changed
- This phase is "pure" with no side effects

Commit Phase:

- React applies the minimal set of changes to the actual DOM
- Executes lifecycle methods or layout effects
- This phase has side effects (DOM mutations)

Key Behaviors:

- React may batch multiple state updates for performance
- Components re-render even if props don't change (unless optimized)
- The entire subtree renders by default when a parent re-renders
- Re-renders don't always cause DOM updates

Optimizing Re-renders:

- Use `React.memo` to memoize functional components
- Implement `shouldComponentUpdate` or extend `PureComponent` for class components
- Use `useMemo` to memoize expensive calculations
- Use `useCallback` to prevent recreating functions

- Optimize context providers to prevent unnecessary consumer updates
- Use proper keys in lists for efficient reconciliation
- Keep state as local as possible to minimize the re-render scope

Common Performance Issues:

- Prop spreading without memoization
- Inline object and function creation in render
- Deeply nested component trees
- Expensive calculations in render methods
- Frequent context updates with many consumers

Debugging Re-renders:

- React DevTools Profiler
- Why-did-you-render library
- Performance monitoring with browser tools

13. How do you test React components?

Testing Libraries:

- **Jest:** JavaScript testing framework (test runner, assertions, mocks)
- **React Testing Library:** Testing utilities focused on user behavior
- **Enzyme:** Component testing focused on implementation details (less common now)
- **Cypress:** End-to-end testing
- **Storybook:** Component development and visual testing

Unit Testing Components:

jsx

 Copy

```
// Using React Testing Library
import { render, screen, fireEvent } from '@testing-library/react';
import Counter from './Counter';

test('Counter increments when button is clicked', () => {
  render(<Counter initialCount={0} />);

  const countDisplay = screen.getByText(/count: 0/i);
  expect(countDisplay).toBeInTheDocument();

  const incrementButton = screen.getByRole('button', { name: /increment/i });
  fireEvent.click(incrementButton);

  expect(screen.getByText(/count: 1/i)).toBeInTheDocument();
});
```

Testing Hooks:

jsx

 Copy

```
// Using React Hooks Testing Library
import { renderHook, act } from '@testing-library/react-hooks';
import useCounter from './useCounter';

test('should increment counter', () => {
  const { result } = renderHook(() => useCounter(0));

  act(() => {
    result.current.increment();
  });

  expect(result.current.count).toBe(1);
});
```

Testing with Context:

jsx

 Copy

```
function renderWithContext(ui, { providerProps, ...renderOptions }) {
  return render(
    <ThemeContext.Provider {...providerProps}>{ui}</ThemeContext.Provider>,
    renderOptions
  );
}

test('shows dark theme content', () => {
  const providerProps = { value: { theme: 'dark' } };
  renderWithContext(<ThemedComponent />, { providerProps });
  expect(screen.getByText(/dark mode/i)).toBeInTheDocument();
});
```

Mocking API Calls:

jsx

 Copy

```
// Using Jest mock functions
jest.mock('../api');

import { fetchUserData } from '../api';
import UserProfile from './UserProfile';

test('displays user data when fetched', async () => {
  fetchUserData.mockResolvedValueOnce({ name: 'John', email: 'john@example.com' });

  render(<UserProfile userId="123" />);

  expect(screen.getByText(/loading/i)).toBeInTheDocument();
  expect(await screen.findByText(/john/i)).toBeInTheDocument();
  expect(fetchUserData).toHaveBeenCalledWith('123');
});
```

Snapshot Testing:

jsx

 Copy

```
test('Button component renders correctly', () => {
  const { container } = render(<Button label="Click me" primary />);
  expect(container).toMatchSnapshot();
});
```

Integration Testing:


```
test('form submission flow works correctly', async () => {
  const mockSubmit = jest.fn();
  render(<ContactForm onSubmit={mockSubmit} />);

  fireEvent.change(screen.getByLabelText(/name/i), {
    target: { value: 'John Doe' }
  });

  fireEvent.change(screen.getByLabelText(/email/i), {
    target: { value: 'john@example.com' }
  });

  fireEvent.click(screen.getByRole('button', { name: /submit/i }));

  expect(mockSubmit).toHaveBeenCalledWith({
    name: 'John Doe',
    email: 'john@example.com'
  });
});
```

Best Practices:

- Test behavior, not implementation details
- Use semantic queries (getByRole, getByLabelText) over implementation-specific ones
- Write tests from a user's perspective
- Implement a testing pyramid (more unit tests, fewer E2E tests)
- Use test-driven development (TDD) when appropriate
- Set up continuous integration for automated testing

14. What strategies do you use for component composition?

Component Types:

- **Container Components:** Handle data fetching, state management
- **Presentational Components:** Focus on UI rendering
- **Compound Components:** Use React.Children and React.cloneElement for related components
- **Higher-Order Components (HOCs):** Wrap components to add functionality
- **Render Props:** Pass rendering logic as a prop function

Composition Patterns:

1. Component Props:

jsx

 Copy

```
function Dialog({ title, content, footer }) {
  return (
    <div className="dialog">
      <header>{title}</header>
      <main>{content}</main>
      <footer>{footer}</footer>
    </div>
  );
}

// Usage
<Dialog
  title={<h2>Welcome</h2>}
  content={<p>This is the main content.</p>}
  footer={<Button>Close</Button>}
/>
```

2. Children Props:

jsx

 Copy

```
function Card({ children, title }) {
  return (
    <div className="card">
      {title} && <div className="card-header">{title}</div>
      <div className="card-body">{children}</div>
    </div>
  );
}

// Usage
<Card title="User Profile">
  <UserDetails user={user} />
  <UserActions user={user} />
</Card>
```

3. Compound Components:

```
function Tabs({ children, defaultIndex = 0 }) {
  const [activeIndex, setActiveIndex] = useState(defaultIndex);

  const contextValue = {
    activeIndex,
    setActiveIndex
  };

  return (
    <TabsContext.Provider value={contextValue}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

function TabList({ children }) {
  return <div className="tab-list">{children}</div>;
}

function Tab({ index, children }) {
  const { activeIndex, setActiveIndex } = useContext(TabsContext);
  return (
    <button
      className={activeIndex === index ? 'active' : ''}
      onClick={() => setActiveIndex(index)}
    >
      {children}
    </button>
  );
}

function TabPanels({ children }) {
  const { activeIndex } = useContext(TabsContext);
  return <div className="tab-panels">{children[activeIndex]}</div>;
}

function TabPanel({ children }) {
  return <div className="tab-panel">{children}</div>;
}

// Compose together
Tabs.TabList = TabList;
Tabs.Tab = Tab;
Tabs.TabPanels = TabPanels;
```

```
Tabs.TabPanel = TabPanel;
```

```
// Usage
```

```
<Tabs defaultIndex={0}>  
  <Tabs.TabList>  
    <Tabs.Tab index={0}>Profile</Tabs.Tab>  
    <Tabs.Tab index={1}>Settings</Tabs.Tab>  
    <Tabs.Tab index={2}>Notifications</Tabs.Tab>  
  </Tabs.TabList>  
  <Tabs.TabPanels>  
    <Tabs.TabPanel>Profile content</Tabs.TabPanel>  
    <Tabs.TabPanel>Settings content</Tabs.TabPanel>  
    <Tabs.TabPanel>Notifications content</Tabs.TabPanel>  
  </Tabs.TabPanels>  
</Tabs>
```

4. Higher-Order Components (HOCs):

```
function withUser(Component) {
  return function WithUser(props) {
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
      fetchUser(props.userId)
        .then(userData => {
          setUser(userData);
          setLoading(false);
        })
        .catch(error => {
          console.error('Error fetching user:', error);
          setLoading(false);
        });
    }, [props.userId]);

    if (loading) return <LoadingSpinner />;
    if (!user) return <ErrorMessage message="User not found" />;

    return <Component {...props} user={user} />;
  };
}

// Usage
const UserProfileWithUser = withUser(UserProfile);
<UserProfileWithUser userId="123" />
```

5. Render Props:

```
function Mouse({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    function handleMouseMove(event) {
      setPosition({
        x: event.clientX,
        y: event.clientY
      });
    }

    window.addEventListener('mousemove', handleMouseMove);
    return () => {
      window.removeEventListener('mousemove', handleMouseMove);
    };
  }, []);

  return render(position);
}

// Usage
<Mouse render={({ x, y }) => (
  <div>
    The mouse position is ({x}, {y})
  </div>
)} />
```

6. Custom Hooks:

```
function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight
  });

  useEffect(() => {
    function handleResize() {
      setSize({
        width: window.innerWidth,
        height: window.innerHeight
      });
    }

    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return size;
}

// Usage in multiple components
function ResponsiveComponent() {
  const { width, height } = useWindowSize();
  return <div>Window size: {width} x {height}</div>;
}
```

Best Practices:

- Favor composition over inheritance
- Keep components small and focused
- Use the right pattern for the specific use case
- Promote reusability and maintainability
- Avoid deeply nested component hierarchies
- Use custom hooks to share stateful logic
- Apply consistent naming conventions

15. How do you handle authentication in React applications?

Authentication State Management:

```
// Context-based authentication
const AuthContext = createContext();

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Check for existing auth token
    const token = localStorage.getItem('auth_token');
    if (token) {
      verifyToken(token)
        .then(userData => {
          setUser(userData);
          setLoading(false);
        })
        .catch(() => {
          localStorage.removeItem('auth_token');
          setLoading(false);
        });
    } else {
      setLoading(false);
    }
  }, []);

  const login = async (credentials) => {
    try {
      const response = await authService.login(credentials);
      localStorage.setItem('auth_token', response.token);
      setUser(response.user);
      return true;
    } catch (error) {
      return false;
    }
  };

  const logout = () => {
    localStorage.removeItem('auth_token');
    setUser(null);
  };

  const value = {
    user,
    loading,
```



```
    login,  
    logout,  
    isAuthenticated: !!user  
  };  
  
  return (  
    <AuthContext.Provider value={value}>  
      {children}  
    </AuthContext.Provider>  
  );  
}  
  
// Hook for consuming the auth context  
function useAuth() {  
  return useContext(AuthContext);  
}
```

Protected Routes:

```
function PrivateRoute({ children }) {
  const { isAuthenticated, loading } = useAuth();
  const location = useLocation();

  if (loading) {
    return <LoadingSpinner />;
  }

  return isAuthenticated ? (
    children
  ) : (
    <Navigate to="/login" state={{ from: location }} replace />
  );
}

// Usage in router setup
<Routes>
  <Route path="/login" element={<LoginPage />} />
  <Route path="/signup" element={<SignupPage />} />
  <Route
    path="/dashboard"
    element={
      <PrivateRoute>
        <Dashboard />
      </PrivateRoute>
    }
  />
</Routes>
```

Authentication Flow:

```
function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const { login } = useAuth();
  const location = useLocation();
  const navigate = useNavigate();

  const from = location.state?.from?.pathname || '/dashboard';

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    try {
      const success = await login({ email, password });
      if (success) {
        navigate(from, { replace: true });
      } else {
        setError('Invalid credentials');
      }
    } catch (err) {
      setError('An error occurred during login');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      {error && <div className="error">{error}</div>}
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        required
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        required
      />
      <button type="submit">Log In</button>
    </form>
  );
}
```

```
);  
}
```

Token Management:

- Store tokens in localStorage or sessionStorage for persistence
- Use HttpOnly cookies for better security
- Implement token refresh mechanisms for expiration
- Add token to HTTP headers for authenticated requests

Authentication With API Middleware:

```
function apiClient() {
  const defaultOptions = {
    headers: {
      'Content-Type': 'application/json',
    },
  };

  return {
    get: (url, options = {}) => {
      const token = localStorage.getItem('auth_token');
      const headers = token
        ? { ...defaultOptions.headers, Authorization: `Bearer ${token}` }
        : defaultOptions.headers;

      return fetch(url, {
        ...defaultOptions,
        ...options,
        headers: {
          ...headers,
          ...options.headers,
        },
        method: 'GET',
      }).then(response => {
        if (response.status === 401) {
          // Handle token expiration
          localStorage.removeItem('auth_token');
          window.location.href = '/login';
          return Promise.reject(new Error('Unauthorized'));
        }
        return response.json();
      });
    },
    // Similar implementations for post, put, delete
  };
}
```

Best Practices:

- Implement proper error handling for authentication failures
- Use secure storage mechanisms for tokens
- Implement token refresh strategies
- Set up proper CORS configurations
- Validate authentication state on both client and server

- Consider using established authentication libraries
- Implement proper logout mechanisms
- Consider role-based access control (RBAC)

Experience-Based Questions

1. Describe a complex React application you've architected.

I designed and led the development of a healthcare analytics platform that processed real-time patient data from multiple sources and presented it through interactive dashboards. This project showcased several architectural challenges and solutions:

Architecture Overview:

- **Frontend:** React with TypeScript, Redux Toolkit for state management
- **Backend:** Node.js microservices with Express, WebSockets for real-time updates
- **Database:** PostgreSQL for relational data, MongoDB for unstructured data
- **Deployment:** Kubernetes orchestration, CI/CD with GitLab

Key Technical Challenges:

1. Real-time Data Processing:

- Implemented a WebSocket-based system to stream data updates to the frontend
- Used Redux middleware to process incoming data streams
- Created custom hooks to abstract WebSocket connection management
- Implemented data throttling and batching for performance

2. Complex State Management:

- Designed a normalized Redux store architecture
- Implemented entity adapters with Redux Toolkit
- Created specialized slices for different domains (patients, metrics, alerts)
- Used selectors for derived data and memoization

3. Performance Optimization:

- Virtualized large data tables and lists with react-window
- Implemented code splitting at route and component levels
- Used React.memo and useMemo strategically for expensive components
- Created a custom image and data caching layer

4. Visualization System:

- Built a composable chart system with D3.js and React
- Implemented a plugin architecture for custom visualizations
- Created a theming system for consistent styling across visualizations
- Optimized rendering performance with canvas for large datasets

5. Authentication and Authorization:

- Implemented JWT-based authentication with refresh tokens
- Created a role-based access control system
- Built a permissions framework for UI component visibility
- Integrated with enterprise SSO systems

6. Accessibility:

- Ensured WCAG 2.1 AA compliance
- Implemented keyboard navigation throughout the application
- Created accessible data visualizations with screen reader support
- Built a color system with sufficient contrast ratios

Project Outcome:

- Successful deployment to 200+ healthcare facilities
- 95% reduction in data processing time compared to previous system
- 40% improvement in user efficiency based on task completion metrics
- Ability to handle 10x the data volume of the previous system

2. How have you migrated legacy React applications to newer patterns?

I led a project to modernize a large-scale e-commerce platform that was built with React 15, class components, and an outdated architecture. Here's how we approached the migration:

Assessment and Planning:

- Conducted a thorough audit of the codebase
- Identified performance bottlenecks and technical debt
- Created a priority list of components for migration
- Established metrics to measure improvements

Incremental Migration Strategy:

- Adopted a "strangler pattern" approach rather than a full rewrite
- Updated the build system first (Webpack, Babel) for modern JavaScript support

- Established a clear boundary between legacy and modern code
- Created adapter components to bridge between systems

Technical Migrations:

1. Class to Functional Components:

- Prioritized leaf components first to minimize ripple effects
- Created HOCs to handle common lifecycle patterns
- Built custom hooks to replace stateful logic
- Implemented React Testing Library tests before migration

2. Redux Migration:

- Moved from Redux+Thunk to Redux Toolkit
- Implemented entity adapters for normalized data
- Refactored thunks to the new createAsyncThunk pattern
- Added proper TypeScript typing for actions and reducers

3. Routing Updates:

- Migrated from React Router v3 to v6
- Implemented code splitting at route boundaries
- Created a backwards-compatible routing layer during transition
- Added proper history management and deep linking

4. Testing Improvements:

- Migrated from Enzyme to React Testing Library
- Implemented proper integration tests for critical flows
- Added Cypress for end-to-end testing
- Integrated testing into CI/CD pipeline

5. Performance Optimizations:

- Implemented React.memo selectively
- Added virtualization for large lists
- Optimized bundle size with code splitting
- Implemented resource hints and lazy loading

Challenges and Solutions:

- **Dependency Management:** Created a dependency upgrade roadmap

- **Knowledge Transfer:** Conducted workshops on modern React patterns
- **Testing Coverage:** Maintained high test coverage during migration
- **Minimal Disruption:** Used feature flags to roll out changes gradually

Results:

- 35% reduction in bundle size
- 50% improvement in initial load time
- 40% reduction in memory usage
- Improved developer productivity and satisfaction
- Successful migration with zero customer-facing issues

3. How do you ensure your React applications are accessible?

Accessibility has been a core focus in my projects. Here's my comprehensive approach:

Project Setup:

- Establish accessibility requirements upfront (typically WCAG 2.1 AA)
- Include people with disabilities in user testing
- Create an accessibility statement and remediation plan
- Implement automated testing in CI/CD pipeline

Development Practices:

1. Semantic Structure:

- Use proper heading hierarchy (`<h1>` through `<h6>`)
- Implement landmark regions (header, main, nav, footer)
- Ensure forms have proper labels and associations
- Use semantic HTML elements over generic divs and spans

2. Keyboard Navigation:

- Ensure all interactive elements are keyboard accessible
- Implement logical tab order (using `tabIndex` when necessary)
- Add visible focus indicators
- Create keyboard shortcuts for complex widgets
- Test all workflows using keyboard only

3. Screen Reader Support:

- Add proper ARIA attributes (roles, states, properties)
- Implement status announcements for dynamic content
- Ensure proper alt text for images
- Use aria-live regions for important updates
- Test with NVDA, JAWS, and VoiceOver

4. Visual Considerations:

- Ensure sufficient color contrast (at least 4.5:1 for normal text)
- Don't rely solely on color to convey information
- Support text resizing up to 200% without loss of functionality
- Ensure content is visible in high contrast mode
- Provide captions and transcripts for multimedia

5. Component-Level Implementation:

- Build an accessible component library
- Create specialized accessible versions of complex widgets:
 - Modal dialogs with proper focus management
 - Dropdown menus with keyboard support
 - Date pickers with screen reader support
 - Data tables with proper headers and ARIA attributes
 - Toast notifications with proper announcements

6. Testing and Validation:

- Automated testing with axe-core and jest-axe
- Manual testing with screen readers
- Keyboard-only testing workflows
- Color contrast validation tools
- Regular accessibility audits

Real-World Example:

For a recent financial dashboard application, I implemented:

- Accessible data tables with proper ARIA attributes and keyboard navigation
- Screen reader announcements for real-time data updates
- Accessible charts with alternative text descriptions and keyboard interaction
- Skip links for navigation

- Proper focus management in modal workflows
- Color-blind friendly visualization palettes
- Full keyboard support for all interactions

The project achieved WCAG 2.1 AA compliance and received positive feedback from users with disabilities, who reported that it was significantly more usable than competing products.

4. Share an example where you significantly improved performance in a React application.

I led a performance optimization initiative for a data-heavy analytics dashboard that was experiencing significant performance issues. Users reported slow initial loads, laggy interactions, and browser crashes with large datasets.

Initial Performance Assessment:

- Used React Profiler to identify problematic components
- Conducted Lighthouse audits to establish baseline metrics
- Used Chrome DevTools Performance panel to identify bottlenecks
- Implemented user timing API to measure real-world performance

Key Issues Identified:

1. Excessive re-renders throughout the component tree
2. Unoptimized bundle size (6MB JavaScript)
3. Large data tables causing layout thrashing
4. Expensive calculations blocking the main thread
5. Inefficient API data fetching patterns

Performance Improvements:

1. Render Optimization:

- Implemented React.memo strategically on heavy components
- Used useCallback for function props to prevent unnecessary rerenders
- Restructured component tree to minimize render scope
- Moved state down to appropriate component levels
- Used useMemo for expensive derived values

Code Example:

```
// Before
function DataRow({ item, onSelect }) {
  const formattedData = expensiveFormatting(item);

  return (
    <tr onClick={() => onSelect(item.id)}>
      <td>{formattedData.name}</td>
      <td>{formattedData.value}</td>
    </tr>
  );
}

// After
const DataRow = React.memo(function DataRow({ item, onSelect }) {
  const formattedData = useMemo(() =>
    expensiveFormatting(item),
    [item]
  );

  const handleClick = useCallback(() =>
    onSelect(item.id),
    [onSelect, item.id]
  );

  return (
    <tr onClick={handleClick}>
      <td>{formattedData.name}</td>
      <td>{formattedData.value}</td>
    </tr>
  );
});
```

2. Data Handling Optimization:

- Implemented virtualized rendering for large data tables
- Created a windowing system for infinite scrolling
- Added data pagination and filtering on the server
- Implemented data caching with React Query
- Used web workers for CPU-intensive calculations

3. Bundle Size Reduction:

- Implemented route-based code splitting

- Removed unused dependencies
- Replaced large libraries with lighter alternatives
- Used dynamic imports for rarely used features
- Optimized SVG assets and implemented proper image loading

4. API and Data Fetching:

- Implemented a GraphQL API to reduce over-fetching
- Added proper data caching and invalidation strategies
- Implemented stale-while-revalidate pattern
- Used batching for related data requests
- Added background prefetching for likely user paths

5. Rendering and Layout:

- Reduced DOM size and nesting depth
- Prevented layout thrashing with read/write batching
- Optimized CSS selectors and reduced specificity
- Implemented CSS containment where appropriate
- Used will-change for elements with frequent animations

Results:

- Initial load time reduced by 73% (5.2s to 1.4s)
- Bundle size reduced by 68% (6MB to 1.9MB)
- Time to interactive improved by 81%
- Memory usage reduced by 60%
- Smooth interactions even with 100K+ data points
- Lighthouse performance score improved from 43 to 92

This performance initiative significantly improved user satisfaction and enabled the application to scale to much larger datasets than previously possible.